TYPE-2 FEEDBACK COMPUTABILITY

J. P. AGUILERA AND M. SOTO

Abstract. Feedback computability studies oracles that provide the convergence and divergence information about computability relative to that same oracle. We consider *strict* and *loose* forms of Feedback Computability relative to Kleene's ${}^2E$, the Type-2 object embodying quantification over natural numbers. We prove that the sets of integers semi-computable this way are precisely those Σ_1-definable over L_{α_I} and $L_{\sigma_1^1}$ respectively, where α_I denotes the least recursively inaccessible ordinal and σ_1^1 the least Σ_1^1-reflecting ordinal. Furthermore, our construction of higher-type feedback computability through ordinal inductive fix-points serves as a natural and general framework for the study of feedback computability.

1. Introduction

Higher-type computability was introduced by Kleene [11] as a generalization of usual, or Type-1, computability. We define objects of Type 0 as all $n \in \mathbb{N}$, and objects of Type $n+1$ as (possibly partial) functions mapping objects of Type n to elements of $\mathbb{N}$. An example of an object of Type 2 is the functional ${}^2E$ given by

$$ {}^2E(x) = \begin{cases} 0, & \text{if } x \text{ is total and } \exists n\, x(n) = 0, \\ 1, & \text{if } x \text{ is total and } \forall n\, x(n) \neq 0, \\ \text{undefined}, & \text{otherwise.} \end{cases} $$

There are natural ways of defining computations in which objects of higher type are used as oracles, e.g., using inductive definability (see Kechris-Moschovakis [10] for an overview). The idea is that, when carrying out a computation $\{e\}^{{}^2E}(n)$, one may allow oracle calls of the following form: apply some subcomputation e' to all natural numbers and some fixed parameters $\vec{p}$ to produce a sequence $\{e'\}(0,\vec{p}), \{e'\}(1,\vec{p}), \{e'\}(2,\vec{p}), \ldots$; then, apply ${}^2E$ to the resulting sequence. Thus, computations may involve infinitely-branching trees of subcomputations (see the example tree on page 4). Indeed, we will say the computation converges if and only if the tree of subcomputations is wellfounded (no infinitely nested oracle calls) and the subcomputation *at each node* only takes finite time (see precise explanations before Definition 1). Computations relative to general Type-2 oracles are defined similarly.

A set of integers x is *semi-recursive* in ${}^2E$ if there is a program e such that

$$ n \in x \leftrightarrow \{e\}^{{}^2E}(n) \downarrow, $$

Date: February 4, 2024 (compiled).

2020 *Mathematics Subject Classification.* 03D60, 03D70, 03D10.

Key words and phrases. feedback, ${}^2E$, oracles, hyperarithmetic computability, hyperjump, least fixed points, admissibility, reflection.

where the $\downarrow$ notation indicates convergence. A set is *recursive* in ${}^2E$ if it and its complement are semi-recursive in ${}^2E$. Kleene showed that the sets of integers recursive in ${}^2E$ are precisely the Δ^1_1 sets ([11]).[1] There is a natural notion of a *halting set* relative to a Type-2 oracle I, namely, the set of all indices e such that $\{e\}^I(e)\downarrow$.

The notion of feedback oracle computability was introduced in [12]. This is a model of computation in which programs are allowed to generate calls to an oracle f, where f is the halting set in the sense of this same model of computation. Such a model allows for nested oracle calls without loss of oracle power. The reason why this does not immediately result in a contradiction by diagonalization is that in this model *oracle calls are not total*: an oracle call may return the answer "yes", it may return the answer "no", or it may return no answer at all, in which case the computation becomes unable to progress and its output is undefined. This is why we define our functions and ${}^2E$ to be partial. The collection of converging computations of this kind can be generated by an inductive definition, much like the collection of converging computations relative to ${}^2E$, and these sets are computably equivalent to each other and Π^1_1-complete.

In this article, we combine both ideas to produce a model of computation of greater strength, and study feedback computability relative to an object of Type 2. We will consider computations in which the program is allowed to generate oracle calls relative to ${}^2E$, as well as to their own halting set. We shall see that there are two natural ways of laying out the details in the definition, and these define models of computation for which the computable sets of integers are those Σ_1-definable over L_{α_I} and $L_{\sigma^1_1}$ respectively, where α_I denotes the least recursively inaccessible ordinal and σ^1_1 the least Σ^1_1-reflecting ordinal. This situation is reminiscent to that of [6], where the notion of *feedback hyperjump* was introduced using ordinal notations. Indeed, our proof of completeness will involve constructing many-one reductions from feedback hyperjumps to feedback Type-2 jumps. Hence, the two notions of feedback computability have the same strength. However, our impression is that casting notions of feedback in terms of oracle computability (rather than in terms of ordinal notations) leads to definitions which are more amenable to further generalization, and in particular naturally enables the study of the ordinal strength of different models of feedback computation in a common general framework. As mentioned in the survey [1] (to which we direct readers interested in the broader picture of feedback computability), a general definition of feedback doesn't yet exist (only its particular applications in different models of computation), and this might be a step in that direction.

2. Feedback Computability with Type-2 Oracles

We describe our model of feedback computability with higher type oracles. We denote by $\langle e\rangle^{{}^2E}$ the program with index e and oracle ${}^2E$. Such programs are

[1] In fact, Kleene's original definition of the Types employed total (instead of partial) functions, and accordingly instead of our ${}^2E$ Kleene defined the following apparently stronger functional:

$$\mathbf{E}(x) = \begin{cases} 0, & \text{if } \exists n\, x(n) = 0, \\ 1, & \text{otherwise} \end{cases}$$

But we will need partiality to make feedback computability work (see next paragraph). And Kleene's proof for $\mathbf{E}$ applies to ${}^2E$ (and also to the ${}^2E^{\#}$ we define in §2), since in fact computability relative to ${}^2E$ easily simulates $\mathbf{E}$.

obtained following the usual Kleene schemata for recursion ([11]), with the following two modifications: First, e is allowed oracle calls to ${}^2E$ as described in §1. Second, e is allowed *feedback calls* to the *feedback halting set*, which we define below.

The feedback halting set will be a function which, when presented with an index e and an input n, outputs

(1) 1, if the computation $\langle e\rangle^{{}^2E}(n)$ converges,
(2) 0, if the computation $\langle e\rangle^{{}^2E}(n)$ diverges,
(3) undefined, if the computation $\langle e\rangle^{{}^2E}(n)$ neither converges nor diverges.

The domain of the feedback halting set *will not* be all of $\mathbb{N}\times\mathbb{N}$, and so it will return nothing (undefined) on some inputs. If a computation $\langle e\rangle^{{}^2E}(n)$ attempts to query the feedback halting set at an undefined input, then the computation cannot continue and we say that $\langle e\rangle^{{}^2E}(n)$ *freezes*.

There are two natural ways of defining this feedback halting set, according to how one defines whether $\langle e\rangle^{{}^2E}(n)$ diverges. With *strict* feedback computability, $\langle e\rangle^{{}^2E}(n)$ diverges if its tree of subcomputations is illfounded or any of them doesn't halt in finite time, but all its appeals to the feedback halting set return an output. With *loose* feedback computability, it is enough for $\langle e\rangle^{{}^2E}(n)$ to diverge that the tree of subcomputations be illfounded or that any of them be non-freezing and not halt, even if the computation $\langle e\rangle^{{}^2E}(n)$ involves freezing subcomputations.

Before continuing, let us remark that, similarly to the distinction between loose and strict feedback computability, we have the distinction between ${}^2E$ and the functional:

$$ {}^2E^\sharp(x) = \begin{cases} 0, & \text{if } \exists n\, x(n) = 0, \\ 1, & \text{if } x \text{ is total and } \forall n\, x(n) \neq 0, \\ \text{undefined}, & \text{otherwise.} \end{cases} $$

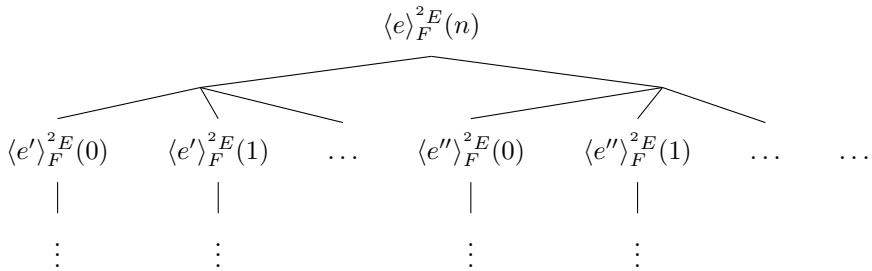
However, our main theorems (Theorem 5 and Theorem 6) remain unchanged if one replaces ${}^2E$ by ${}^2E^\sharp$ throughout. This is because, on the one hand, ${}^2E^\sharp$ computes ${}^2E$ (as is well known) and, on the other, the upper bounds we prove in §3 remain true for the notions of feedback computability relative to ${}^2E^\sharp$ (by the same proofs). It is similarly easy to prove (by simulation; see footnote 1) that the reals computable in ${}^2E$ and in ${}^2E^\sharp$ are the same. In fact, by a theorem of Kleene [11] these are precisely the Δ^1_1, or *hyperarithmetical*, reals.

We now turn to precise definitions. First, we define the notion of feedback computability relative to ${}^2E$ and a *feedback oracle* F. A feedback oracle is a partial function

$$\mathbb{N}\times\mathbb{N}\to\{\text{yes, no}\}.$$

Computations $\langle e\rangle^{{}^2E}_F(n)$ are defined as before, except that feedback calls are resolved according to F. If a call is made to F but the input is not in the domain of F, the program *freezes*.

Given a computation $\langle e\rangle^{{}^2E}_F(n)$, the subcomputation tree $T_{e,n,F}$ of $\langle e\rangle^{{}^2E}_F(n)$ is defined the usual way: if any subcomputation makes use of an oracle call to ${}^2E$ with input e', then the infinitely many nodes $\langle e'\rangle^{{}^2E}_F(k)$, for $k\in\mathbb{N}$, are placed below said subcomputation (trees grow downwards). Thus, oracle calls to ${}^2E$ give rise to infinitely many immediate subcomputations (child nodes), while oracle calls to F give rise to no immediate subcomputations.



We imagine the computation as routing through the tree of oracle calls, eventually returning to the root to finish the whole computation if it converges. For the next definition, by the computation being finite *at one particular node* we mean the computation defined by the label of said node halts after a finite number of steps, *without* taking into account the steps performed by the subcomputations it calls through oracle queries (that is, those labeled in child nodes). That is, we count how many steps occur when the overall computation is "at that node" (not in any parent nor child, that is, within outer or inner oracle queries).
For example, a computation might run for 5 steps, the last of which is an oracle query to ${}^2E$ spawning infinitely many subcomputations (each of which doesn't make any further oracle calls, and runs for some finite number of steps), and then given the answer to that query runs for 3 more steps and halts. We would then say that particular computation (corresponding to the root of the tree) has performed 8 steps (even though the subcomputations called by the oracle query, corresponding to the children nodes, might have performed many more).
A way in which this finiteness might fail is by the computation in that node being just a classically non-halting computation (with no further oracle queries). But it can also happen that the computation places infinitely many different sequential (not nested) calls to ${}^2E$, so that even if all of them return an answer satisfactorily, the main computation never halts.

Definition 1. Let e, n and F be as above.

(1) The computation $\langle e\rangle_F^{{}^2E}(n)$ *converges* if $T_{e,n,F}$ is wellfounded, and the number of computational steps *at each node* (as defined above) is finite.
(2) The computation $\langle e\rangle_F^{{}^2E}(n)$ *strictly diverges* if all appeals to F in any node of $T_{e,n,F}$ (that is, at any point during the whole computation) return an answer, but either $T_{e,n,F}$ is illfounded, or the computation *at some node* runs for infinitely many steps.
(3) The computation $\langle e\rangle_F^{{}^2E}(n)$ *diverges* or *loosely diverges* if either $T_{e,n,F}$ is illfounded, or the computation *at some node* runs for infinitely many steps.

We can now define the *strict feedback operator* Φ_S. This operator maps feedback oracles to feedback oracles and is defined as follows:

$$\Phi_S(F)(e,n) = \begin{cases} 1, & \text{if } \langle e\rangle_F^{{}^2E}(n) \text{ converges,} \\ 0, & \text{if } \langle e\rangle_F^{{}^2E}(n) \text{ strictly diverges,} \\ \text{undefined,} & \text{otherwise.} \end{cases}$$

Similarly, the *loose feedback operator* is defined by

$$\Phi_L(F)(e,n) = \begin{cases} 1, & \text{if } \langle e\rangle_F^{{}^2E}(n) \text{ converges,} \\ 0, & \text{if } \langle e\rangle_F^{{}^2E}(n) \text{ loosely diverges,} \\ \text{undefined,} & \text{otherwise.} \end{cases}$$

Lemma 2. *Φ_S and Φ_L are monotone, i.e., if F is an extension of G, then $\Phi_S(F)$ is an extension of $\Phi_S(G)$, and similarly for Φ_L.*

Proof. Clear from the definition. □

These operators can be iterated by setting

$$\Phi_S^\alpha = \Phi_S(\Phi_S^{<\alpha}), \text{ where } \Phi_S^{<\alpha} = \bigcup_{\beta<\alpha} \Phi_S^\beta$$

$$\Phi_S^\infty = \bigcup_\alpha \Phi_S^\alpha$$

Similarly for Φ_L.

Definition 3. The *strict ${}^2E$-feedback halting set* ${}^2\mathcal{S}$ is defined as the graph of the inductive fixpoint Φ_S^∞. The *loose ${}^2E$-feedback halting set* ${}^2\mathcal{L}$ is defined as the graph of the inductive fixpoint Φ_L^∞.

Strict and loose feedback computability relative to ${}^2E$ are defined by

$$\langle e\rangle_{\mathcal{S}}^{{}^2E}(n) := \langle e\rangle_{{}^2\mathcal{S}}^{{}^2E}(n),$$

$$\langle e\rangle_{\mathcal{L}}^{{}^2E}(n) := \langle e\rangle_{{}^2\mathcal{L}}^{{}^2E}(n)\ {}^2$$

Our goal is to study the sets of integers which can be computed by strict and loose feedback computability relative to ${}^2E$. Let us make this more precise:

Definition 4. Let $x \subset \mathbb{N}$. We say x is *strict feedback semi-computable from ${}^2E$* if and only if there exists $e \in \mathbb{N}$ such that

$$\forall k \left(k \in x \leftrightarrow \langle e\rangle_{\mathcal{S}}^{{}^2E}(k) \downarrow\right)$$

Equivalently, if and only if there exists $e \in \mathbb{N}$ such that

$$\forall k \left(k \in x \leftrightarrow \langle e\rangle_{\mathcal{S}}^{{}^2E}(k) = 1\right)$$

Similarly for *loose*.
We say that x is *strict* (respectively, *loose*) *feedback computable from ${}^2E$* if and only if both x and $\mathbb{N} \setminus x$ are strict (respectively, loose) feedback semi-computable from ${}^2E$.

Our goal is to prove the following result for strict feedback:

Theorem 5. *The following are equivalent:*

(1) x is strict feedback semi-computable from ${}^2E$, and
(2) x is Σ_1-definable over L_{α_I}, where α_I is the least recursively inaccessible ordinal.

[2]We employ both superscripts and subscripts in our notation to emphasize the different roles the two oracles are performing, which play into the construction differently, and accordingly can be represented by two different tree strcutures (more on this below).

Here, recall that α is recursively inaccessible if and only if it is both admissible and a limit of admissibles. And recall as well that an admissible ordinal is an α such that L_α is an admissible set, that is, a transitive model of Kripke-Platek set theory (see [8]). This happens exactly when α is a limit and $L_\alpha \models \Sigma_0-$Collection.

Similarly, we will have the following result for loose feedback:

Theorem 6. *The following are equivalent:*

(1) x is loose feedback semi-computable from ${}^2E$, and
(2) x is Σ_1-definable over $L_{\sigma_1^1}$, where σ_1^1 is the least Σ_1^1-reflecting ordinal.

Here, recall that α is Σ_1^1-reflecting if whenever $\phi(x)$ is a (second-order) Σ_1^1 formula in the language of set theory and $\beta < \alpha$, we have

$$L_\alpha \models \phi(\beta) \to \exists \bar{\alpha} < \alpha \left(\beta < \bar{\alpha} \wedge L_{\bar{\alpha}} \models \phi(\beta)\right).$$

3. Upper bounds for Theorems 5 and 6

Definition 7. The *strict rank* (denoted $\rho_S(e,n)$) of a pair (e,n) is the least ordinal α such that (e,n) belongs to the domain of Φ_S^α, if such an α exists; otherwise, we define the strict rank of (e,n) to be ∞. The *loose rank* (denoted $\rho_L(e,n)$) of (e,n) is defined analogously.

The following lemma yields the upper bound for Theorem 6. We have chosen to prove Lemma 8 by a short argument involving various results from the literature on Higher Recursion Theory to showcase these connections, but we remark that it could also be proven directly by reasoning similar to that of the proof of Lemma 10 below, by an argument similar to the one in [6].

Lemma 8. *Suppose that $\rho_L(e,n) < \infty$. Then $\rho_L(e,n) < \sigma_1^1$.*

Proof. Observe that the operator $F \mapsto \Phi_L(F)$ is given by the union of a Π_1^1 operator and a Σ_1^1 operator. The Π_1^1 operator adds points to the domain of the operator in the preimage of $\{1\}$. Note that the convergence of $\langle e \rangle_F^{{}^2E}(n)$ is indeed a $\Pi_1^1(F)$ statement, uniformly in F; namely, one expressing the wellfoundedness of a tree of subcomputations. The Σ_1^1 operator adds points to the domain of the operator in the preimage of $\{0\}$. Without defining the classes of relations involved in the argument below, let us mention that by a theorem of Aczel-Richter [3], we have

$$\Sigma_1^{L_{\sigma_1^1}} = \Sigma_1^1\text{–}\mathsf{IND},$$

i.e., the sets of integers Σ_1-definable over $L_{\sigma_1^1}$ are precisely those definable by fixpoints of Σ_1^1-operators on $\mathbb{N}$. By the companion theorem of Moschovakis [13], $\Sigma_1^{L_{\sigma_1^1}}$ is what is known as a *Spector class*. We shall not define this here, but note that by the First Recursion Theorem of Moschovakis (see Kechris [9, Theorem 1.7]), in order to see that the fixpoint of Φ_L is in $\Sigma_1^{L_{\sigma_1^1}}$, it suffices to check that $\Sigma_1^{L_{\sigma_1^1}}$ is closed under Φ_L. But $\Sigma_1^{L_{\sigma_1^1}}$ is closed under illfoundedness by Σ_1^1-reflection and $\Sigma_1^{L_{\sigma_1^1}}$ is closed under wellfoundedness because every Spector class is (this follows from a theorem of Aczel; see [5] for more details). Therefore, we have

$$\Phi_L^\infty = \Phi_L^{\sigma_1^1}$$

and hence the conclusion of the lemma follows. □

We remark that the proof given above can be applied to computability relative to any Δ^1_1-definable Type-2 functional, and not only to ${}^2E$.

Corollary 9. *If a set of integers x is loose feedback semi-computable from ${}^2E$, then it is Σ_1-definable over $L_{\sigma^1_1}$.*

Proof. Suppose x is loose feedback semi-computable from ${}^2E$. Then, for some index e, we have for each $k \in \mathbb{N}$:

$$\begin{aligned} k \in x &\leftrightarrow \langle e \rangle^{{}^2E}_{\mathcal{L}}(k) \downarrow \\ &\leftrightarrow \exists \alpha < \infty \big(\Phi^{\alpha}_L(e,k) = 1\big) \\ &\leftrightarrow \exists \alpha < \sigma^1_1 \left(\Phi^{\alpha}_L(e,k) = 1\right) \end{aligned}$$

as needed. □

We will now prove the analog to Lemma 8 for ${}^2\mathcal{S}$, which we do more directly instead of invoking other results.

Lemma 10. *Suppose that $\rho_S(e,n) < \infty$. Then $\rho_S(e,n) < \alpha_I$.*

Proof. We will see all witnesses for convergence and divergence for ${}^2\mathcal{S}$ belong to L_{α_I}. Hence, we will have $\Phi^{\alpha_I}_S \subseteq \Phi^{<\alpha_I}_S$.

We consider a (non-freezing) program $\langle e \rangle^{{}^2E}_{\Phi^{<\alpha_I}_S}$ run on some input n. The main observation is that we are able to build $T_{e,n,{}^2\mathcal{S}}$ one level at a time (downwards) and prove it will belong to L_{α_I}. For every call to ${}^2E$, all subcomputations called upon will be non-freezing. Thus, for each m, the mth level of $T_{e,n,{}^2\mathcal{S}}$ can be uniformly constructed in L_{α_I}, so that by admissibility $T_{e,n,{}^2\mathcal{S}} \in L_{\alpha_I}$. Consider the function f mapping each node c in the tree of subcomputations to the least stage γ such that $\Phi^{<\gamma}_S$ returns an input to all feedback oracle calls in c (and in its subcomputations). Since f is a total function from a set in L_{α_I} to α_I, it follows by the admissibility of α_I that the range of f is bounded, say by some $\delta < \alpha_I$.

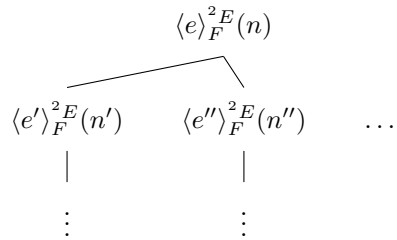
Since all feedback oracle calls in $\langle e \rangle^{{}^2E}_{\Phi^{<\alpha_I}_S}$ already return an answer when prompting the oracle $\Phi^{<\delta}_S$, the convergence of $\langle e \rangle^{{}^2E}_{\Phi^{<\alpha_I}_S}$ is witnessed by a ranking function of $T_{e,n,{}^2\mathcal{S}}$, which belongs to any admissible set containing $T_{e,n,{}^2\mathcal{S}}$ and $\Phi^{<\delta}_S$. Since α_I is a limit of admissibles, such a set belongs to L_{α_I}.

As for divergence: regular divergence ($T_{e,n,{}^2\mathcal{S}}$ is well-founded but some subprogram runs for infinitely many steps) can be trivially checked from $T_{e,n,{}^2\mathcal{S}}$. The divergence of computations with $T_{e,n,{}^2\mathcal{S}}$ ill-founded (infinite nested calls to ${}^2E$) will be witnessed by an infinite descending chain through $T_{e,n,{}^2\mathcal{S}}$, which will be definable over any admissible set containing $T_{e,n,{}^2\mathcal{S}}$ and $\Phi^{<\delta}_S$. As before, since α_I is a limit of admissibles, such a sequence belongs to L_{α_I}. □

Corollary 11. *If a set of integers x is strict feedback semi-computable from ${}^2E$, then it is Σ_1-definable over L_{α_I}.*

Proof. Like Corollary 9. □

The following result will be useful for the proof of Theorem 14 below. In contrast to the subcomputation tree $T_{e,n,F}$, define now the *oracle tree* $t_{e,n,F}$, which includes as subcomputations the calls to F (generating a single node immediately below), and not those to ${}^2E$.



Lemma 12. *$\rho_S(e,n)=\infty \iff t_{e,n,{}^2\mathcal{S}}$ is ill-founded.*

Proof. Inductively on the rank of $t_{e,n,{}^2\mathcal{S}}$, if it is well-founded then $\langle e\rangle_{\mathcal{S}}^{{}^2E}(n)$ doesn't freeze. The converse hinges on ${}^2\mathcal{S}$ being the least fixed point. Let's see

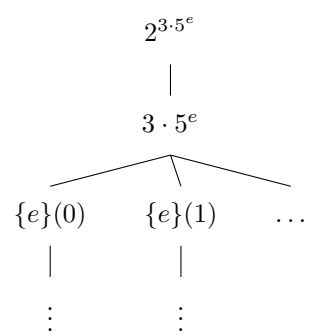
$$W = {}^2\mathcal{S} \restriction \{(e,n,1/0) \mid t_{e,n,{}^2\mathcal{S}} \text{ is well-founded}\}$$

is a fixed point. Suppose $\Phi_S(W)(e,n)$ returns an answer. Since $\langle e\rangle_W^{{}^2E}(n)$ doesn't freeze, any oracle call made by this computation must be on a pair (e',n') in the domain of W. By definition of W, for each of these (e',n') the tree $t_{e',n',{}^2\mathcal{S}}$ is well-founded. Thus $t_{e,n,{}^2\mathcal{S}}$ also is. We thus have ${}^2\mathcal{S}\subseteq W$, as needed. □

4. Lower bounds for Theorems 5 and 6

In order to prove the lower bounds for Theorems 5 and 6, we shall need some preliminaries; namely, we will need the definition of the feedback hyperjump from [6]. Our plan will be to find suitable reductions from feedback hyperjumps to ${}^2\mathcal{S}$ and ${}^2\mathcal{L}$ and appeal to the results of [6]. We also will make use of Kleene's recursion theorem (whose proof for feedback computability is the same as for ordinary computability): for every partial recursive function f, there is an index e for a program such that for all inputs n, $\{e\}(n)=\{f(e)\}(n)$.

4.1. **Feedback hyperjump.** Building upon Kleene's $\mathcal{O}$ we'll define $\mathcal{SO}$ and $\mathcal{LO}$, which harness further computational power through their inductive definition, including programs consulting oracles.
It is natural to define a tree T_n, which intuitively witnesses the construction of n as an ordinal notation, so that $n\in\mathcal{O}$ iff T_n is well-founded and satisfies some additional properties (what we call *well-formed*, see [6, Definition 1] for more details). To build T_n, we place n at the root, and then iteratively give every node labeled 2^m a child labeled m, and every node labeled $3\cdot 5^e$ children $\{e\}(k)$ (maybe undefined).



Similarly, we can define another tree of oracle notation U_n^P which also witnesses positively the failure to be an ordinal notation when using an oracle P in our computations. We build U_n^P just as above, but now invoking oracle computations $\{e\}^P(k)$ (see [6, Definition 2] for more details). U_n^P is necessary to construct our reflective oracles $\mathcal{SO}$ and $\mathcal{LO}$, which include this positive information regarding failure about themselves.
Note that $\mathcal{O}$ answers questions both of the form $n \in \mathcal{O}?$ and $n <_{\mathcal{O}} m?$. The below oracles, which approximate $\mathcal{SO}$ and $\mathcal{LO}$, also do.

Definition 13. Given an oracle P, we define the augmented oracle P^+ as

(i) $P^+(n \in \mathcal{SO}?) = Y$ if U_n^P is well-founded and well-formed
(ii) $P^+(n \in \mathcal{SO}?) = N$ if U_n^P is either non-freezing and ill-formed, or is well-formed (and thus also non-freezing) and ill-founded
(iii) $P^+(n <_{\mathcal{SO}} m?) = Y$ if $P^+(n \in \mathcal{SO}?) = Y$, $m \neq n$ and m is a node of U_n^P
(iv) $P^+(n <_{\mathcal{SO}} m?) = N$ if either $P^+(n \in \mathcal{SO}?) = N$, or $P^+(n \in \mathcal{SO}?) = Y$ and $m = n$ or m is not a node of U_n^P

Since this definition is positive in P, we can define $\mathcal{SO}$ or *strict* $\mathcal{O}$ to be the least fixed point of the operation $P \mapsto P^+$.
If instead in (ii) we only require that U_n^P be either ill-formed or ill-founded (and so possibly freezing), we obtain $\mathcal{LO}$ or *loose* $\mathcal{O}$.

4.2. **Proof of the lower bounds.**

Theorem 14. *There exist computable functions f and g such that*

(i) ${}^2\mathcal{S}(f(n)) = 1 \iff \mathcal{SO}(n \in \mathcal{SO}?) = Y$
(ii) ${}^2\mathcal{S}(f(n)) = 0 \iff \mathcal{SO}(n \in \mathcal{SO}?) = N$
(iii) ${}^2\mathcal{S}(g(m,n)) = 1 \iff \mathcal{SO}(m <_{\mathcal{SO}} n?) = Y$
(iv) ${}^2\mathcal{S}(g(m,n)) = 0 \iff \mathcal{SO}(m <_{\mathcal{SO}} n?) = N$

And thus also

(v) $f(n) \notin dom({}^2\mathcal{S}) \iff \mathcal{SO}(n \in \mathcal{SO}?)$ *is undefined*
(vi) $g(m,n) \notin dom({}^2\mathcal{S}) \iff \mathcal{SO}(m <_{\mathcal{SO}} n?)$ *is undefined*

Similarly for ${}^2\mathcal{L}$ and $\mathcal{LO}$ (with the same f and g).

Proof. Intuitively, $f(n)$ will be a program trying to compute whether n is a notation. We present the recursive definitions of f and g, which are interdependent on each other. They will be guaranteed to exist and be computable by Kleene's recursion theorem (since they need to call upon themselves).

Take c and d any fixed programs that, respectively, converge and diverge without calling any of the two oracles. These programs will be used in the definition of f and g in order to ensure that ${}^2\mathcal{S}$ attains specific values when applied to $f(n)$ or $g(m,n)$. Thus, we do not care about the output of c, but only about the fact that it converges. We describe the function f informally:

- For n not of the form 2^m or $3 \cdot 5^e$, $f(n) = d$
- $f(1) = c$
- $f(2^m) = f(m)$ for $m \neq 0$
- $f(3 \cdot 5^e)$ is a program calling ${}^2E$ on another program e' functionally defined below. This is the call that does most of the work, by harnessing the infinite calls of ${}^2E$ to evaluate infinitely branching nodes of the notation tree. e' acts as follows:

($e'1$) When run on input k (in one of the infinitely many calls made by ${}^2E$), e' first simulates the runtime of $\{e\}^{\mathcal{SO}}(k)$ to obtain its output. That is, it runs all of the usual Turing procedures by transforming them into arithmetical relations as per the usual trick, and whenever $\{e\}^{\mathcal{SO}}(k)$ makes the oracle call $n \in \mathcal{SO}$? for a certain n, our e' instead computes $f(n)$ and calls the oracle ${}^2\mathcal{S}(f(n))$.

($e'2$) Having noted this output (if the computation converged), it does the same for $\{e\}^{\mathcal{SO}}(k+1)$ using the rest of the tape.

($e'3$) To check whether these outputs are notations, it will compute their images by f and ask ${}^2\mathcal{S}(f(\{e\}^{\mathcal{SO}}(k)))$ and ${}^2\mathcal{S}(f(\{e\}^{\mathcal{SO}}(k+1)))$. If the answer to any of the two is negative, run d. Otherwise, continue.

($e'4$) Finally, to check whether $\{e\}^{\mathcal{SO}}(k) <_{\mathcal{SO}} \{e\}^{\mathcal{SO}}(k+1)$, it computes the image of said pair by g and calls on ${}^2\mathcal{S}(g(\{e\}^{\mathcal{SO}}(k), \{e\}^{\mathcal{SO}}(k+1)))$ (and runs c or d accordingly).

Complementarily, the program $g(m,n)$ is functionally defined as follows:

($g1$) $g(m,n)$ first calls ${}^2\mathcal{S}(f(n))$, and then checks whether $m \neq n$. If any of the two is negative, run d.

($g2$) Otherwise, it will check whether m is a node in $U_n^{\mathcal{SO}}$. For that, it starts computing said tree, and after any one of its steps (computing the numerical value of a new node), checks whether it coincides with m.
In these steps, as per the definition of $U_n^{\mathcal{SO}}$, it will start with n, and it will turn any number of the form 2^r into r. And for any number of the form $3 \cdot 5^r$, it will do the following: by dovetailing, run simultaneously for all k the program that first computes the output $\{r\}^{\mathcal{SO}}(k)$ (by simulating said computation as explained in ($e'1$) above), and then keeps on checking whether m is a node in $U^{\mathcal{SO}}_{\{r\}^{\mathcal{SO}}(k)}$, that is, applying again the whole of step ($g2$) with a new n.

($g3$) If at any point the coincidence is found, run c. If the search ends and no coincidence is found, run d. If the search doesn't end, of course it will automatically diverge.

Given these definitions, the equivalence of strict convergence follows by induction on the construction of $U_n^{\mathcal{SO}}$. And for that of strict freezing we can see $f(n)$ makes infinite nested calls to ${}^2\mathcal{S}$ (which is a characterization of freezing by Lemma 12) iff some program called in $U_n^{\mathcal{SO}}$ does so for $\mathcal{SO}$ (which is a characterization of freezing by [6, Proposition 8]). Similarly it is straightforward to check loose convergence and loose divergence (instead of freezing) by considering the definitions of both $U_n^{\mathcal{LO}}$ and $f(n)$ (and the trees $T_{e,k,{}^2\mathcal{L}}$). □

5. Further questions

Our work in this article characterizes the sets of integers which are feedback-computable relative to the Type-2 oracle ${}^2E$, according to both the *strict* and *loose* notions of feedback computability. Our definitions of feedback readily serve as a framework for the general study of the feedback strength of Type-n functionals.

It is thus natural to consider feedback computability relative to other oracles. For example, is there a characterization of the sets of integers feedback-computable from Cenzer's [7] functional $G_1^\sharp$ or Hinman's functional $E_1^\sharp$? (See Aczel [2] for a definition of the latter; recall that the sets of integers semi-computable from $G_1^\sharp$

and $E_1^\sharp$ are precisely those which are Σ_1 over $L_{\sigma_1^1}$ and $L_{\pi_1^1}$, respectively.) What about the *superjump* (see Aczel-Hinman [4])?

One can also drop particular functionals and ask more generally about complexity classes of them. For example, is there a characterization of the sets of integers feedback-computable from *some* Σ_2^1 Type-2 functional?

References

[1] N. L. Ackerman, C. E. Freer, R. S. Lubarsky. An introduction to feedback Turing computability. In *Journal of Logic and Computation*, Volume 30, Issue 1, Pages 27–60, 2020.

[2] P. Aczel. Representability in Some Systems of Second Order Arithmetic. *Israel J. Math.*, pages 309–328. 1970.

[3] P. Aczel and W. Richter. Inductive Definitions and Reflecting Properties of Admissible Ordinals. In J. E. Fenstad and P. G. Hinman, editors, *Generalized Recursion Theory*, pages 301–381. 1974.

[4] P. Aczel and P.G. Hinman. Recursion in the Superjump. In J. E. Fenstad and P. G. Hinman, editors, *Generalized Recursion Theory*, pages 3–41. 1974.

[5] J. P. Aguilera and F. V. Kerchove. A proof of Grilliot's theorem. Forthcoming.

[6] J. P. Aguilera and R. Lubarsky. Feedback hyperjump. *J. Log. Comput.*, 31:20–39, 2021.

[7] D. Cenzer. Ordinal Recursion and Inductive Definitions. In J. E. Fenstad and P. G. Hinman, editors, *Generalized Recursion Theory*, pages 221–264. 1974.

[8] K. J. Devlin. *Constructibility*. Perspectives in Logic, Volume 6, 1984.

[9] A. S. Kechris. On Spector Classes. In A. S. Kechris, B. Löwe, and J. R. Steel, editors, *Ordinal definability and recursion theory, The Cabal Seminar, Volume III*. Cambridge University Press, 2016.

[10] A. S. Kechris and Y. N. Moschovakis. Recursion in Higher Types. In J. Barwise, editor, *Handbook of Mathematical Logic*, pages 681–737. 1977.

[11] S. C. Kleene. Recursive Functionals and Quantifiers of Finite Types, I. *Trans. Amer. Math. Soc.*, 91:1–52, 1959.

[12] R. S. Lubarsky. ITTMs with Feedback. In R. Schindler, editor, *Ways of Proof Theory*, pages 341-354. Ontos, 2010.

[13] Y. N. Moschovakis. *Elementary Induction on Abstract Structures*. Studies in Logic and the Foundations of Mathematics. Elsevier, 1974.